

QATrack — Dokumentasi Teknis Sistem Notifikasi & Inti Alur Kode

Panduan Lengkap Arsitektur, Alur Role (Super Admin, Support QA, Developer), dan Potongan Kode per File

1. Akun Login & Hak Akses Role

Sistem QATrack menggunakan sistem autentikasi multi-role yang terhubung langsung ke notifikasi. Seluruh akun default menggunakan password: **pisangkeju**.

Role	Akun / Email	Hak Akses & Notifikasi yang Diterima
Super Admin	masteryoda yoda@jedi.com	Akses penuh sistem, manajemen user (CRUD), menerima notifikasi seluruh tiket baru, penugasan developer, status perbaikan bug, dan log aktivitas.
Support Dev (QA)	obiwan obiwan@jedi.com	Melaporkan bug baru, melihat tiket laporannya, menerima notifikasi ketika developer mulai mengerjakan atau menandai bug selesai (siap retest).
Developer	anakin anakin@jedi.com	Melihat tiket yang ditugaskan kepadanya, menerima notifikasi penugasan tiket baru, alert darurat tiket kritis (P1), dan mengubah status pengerjaan (In Progress -> Fixed).

2. Inti Alur Sistem Notifikasi Terhubung ke Semua Role

Alur 1: Pelaporan Bug Baru (Bug Creation)

- Support Dev/Admin mengisi form di bugs-create.blade.php -> Dikirim via POST /bugs.
- BugController@store memvalidasi data dan menyimpan tiket ke tabel bugs.
- Sistem menembakkan BugNotification: Super Admin menerima kabar tiket masuk, Developer menerima notifikasi penugasan, dan seluruh Developer menerima peringatan darurat jika tiket berprioritas P1.

Alur 2: Pembaruan Status & Penugasan (Status / Assignee Update)

- Developer atau Admin menekan tombol status di bugs-show.blade.php -> Dikirim via PATCH /bugs/{bug}/status.
- BugController@updateStatus mengecek hak status dan memperbarui field di database.
- Sistem mendistribusikan notifikasi: Pelapor bug (Support Dev) diberi tahu progres pengerjaan, Developer diberi tahu jika ada perubahan tugas/status oleh tim lain, dan Admin menerima log riwayat aktivitas.

Alur 3: Antarmuka Lonceng Header (Header Bell UI & AJAX)

- Header menghitung unread_count secara server-side. Dot merah dengan animasi pulse (animate-ping) menyala jika ada unread > 0.
- Pengguna dapat mengklik ikon lonceng untuk membuka panel dropdown responsif dengan tab 'Semua' dan 'Belum Dibaca'.
- Mengklik item notifikasi otomatis menandainya telah dibaca via AJAX (POST /notifications/{id}/read) dan langsung mengarahkan browser ke detail bug terkait.
- Background polling setiap 30 detik memanggil GET /notifications untuk menyinkronkan status badge tanpa reload.

3. Potongan Kode Inti per File yang Digunakan

Berikut adalah potongan kode kunci dari setiap komponen arsitektur sistem:

File: routes/web.php — *Routing Notifikasi & Tiket*

Fungsi Inti: Mendaftarkan endpoint API notifikasi dan tiket bug dalam proteksi autentikasi.

```
// routes/web.php
Route::middleware('auth')->group(function () {
    // Endpoints Notifikasi
    Route::get('/notifications', [NotificationController::class, 'index'])->name('notifications.index');
    Route::post('/notifications/{id}/read', [NotificationController::class, 'markAsRead'])->name('notifications.read');
    Route::post('/notifications/read-all', [NotificationController::class, 'markAllAsRead'])->name('notifications.read-all');
    Route::delete('/notifications/{id}', [NotificationController::class, 'destroy'])->name('notifications.destroy');
    Route::delete('/notifications', [NotificationController::class, 'clearAll'])->name('notifications.clear-all');

    // Endpoints Bug & Status
    Route::get('/bugs', [BugController::class, 'index'])->name('bugs');
    Route::post('/bugs', [BugController::class, 'store'])->name('bugs.store');
    Route::get('/bugs/{bug}', [BugController::class, 'show'])->name('bugs.show');
    Route::patch('/bugs/{bug}/status', [BugController::class, 'updateStatus'])->name('bugs.status.update');
});
```

File: app/Notifications/BugNotification.php — *Kelas Notifikasi Database*

Fungsi Inti: Mendefinisikan payload data notifikasi terstruktur (judul, pesan, tipe, URL bug, ikon, dan warna badge).

```
// app/Notifications/BugNotification.php
class BugNotification extends Notification
{
    use Queueable;

    public function __construct(
        protected string $title,
        protected string $message,
        protected string $type = 'bug',
        protected ?int $bugId = null,
        protected ?string $url = null,
        protected string $icon = 'notifications',
        protected string $badgeColor = 'primary'
    ) {}

    public function via(object $notifiable): array {
        return ['database']; // Disimpan ke tabel 'notifications'
    }

    public function toArray(object $notifiable): array {
        return [
            'title' => $this->title,
            'message' => $this->message,
            'type' => $this->type,
            'bug_id' => $this->bugId,
            'url' => $this->url ?? ($this->bugId ? route('bugs.show', $this->bugId) : null),
            'icon' => $this->icon,
            'badge_color' => $this->badgeColor,
        ];
    }
}
```

File: app/Http/Controllers/NotificationController.php — Controller Notifikasi

Fungsi Inti: Menyediakan endpoint JSON untuk melayani antarmuka dropdown lonceng dan aksi mark-as-read.

```
// app/Http/Controllers/NotificationController.php
class NotificationController extends Controller
{
    public function index(Request $request): JsonResponse {
        $user = $request->user();
        $filter = $request->query('filter', 'all');
        $query = $user->notifications();
        if ($filter === 'unread') { $query->whereNull('read_at'); }

        $notifications = $query->latest()->take(20)->get()->map(function ($notif) {
            return [
                'id' => $notif->id,
                'title' => $notif->data['title'] ?? 'Notifikasi',
                'message' => $notif->data['message'] ?? '',
                'url' => $notif->data['url'] ?? null,
                'icon' => $notif->data['icon'] ?? 'notifications',
                'badge_color' => $notif->data['badge_color'] ?? 'primary',
                'is_read' => $notif->read_at !== null,
                'time_ago' => $notif->created_at ? $notif->created_at->diffForHumans() : '',
            ];
        });

        return response()->json([
            'notifications' => $notifications,
            'unread_count' => $user->unreadNotifications()->count(),
            'total_count' => $user->notifications()->count(),
        ]);
    }

    public function markAsRead(Request $request, string $id): JsonResponse {
        $notif = $request->user()->notifications()->where('id', $id)->firstOrFail();
        $notif->markAsRead();
        return response()->json(['success' => true, 'unread_count' => $request->user()->unreadNotifications()->count()]);
    }

    public function markAllAsRead(Request $request): JsonResponse {
        $request->user()->unreadNotifications->markAsRead();
        return response()->json(['success' => true, 'unread_count' => 0]);
    }
}
```

File: app/Http/Controllers/BugController.php — Logika Notifikasi di BugController

Fungsi Inti: Memicu distribusi notifikasi otomatis saat bug baru dibuat atau status pengerjaan tiket berubah.

```
// app/Http/Controllers/BugController.php (Potongan store & updateStatus)
public function store(Request $request)
{
    $validated = $request->validate([...]);
    $validated['reporter_id'] = $request->user()->id;
    $bug = Bug::create($validated);

    // 1. Notifikasi ke Super Admin
    $superAdmins = User::where('role', 'super_admin')->where('id', '!=', $request->user()->id)->get();
    foreach ($superAdmins as $admin) {
        $admin->notify(new BugNotification(
            title: 'Bug Baru Dilaporkan',
            message: "{$request->user()->name} melaporkan bug #{$bug->id}: '{$bug->title}'",
            type: 'bug_created', bugId: $bug->id, icon: 'bug_report',
            badgeColor: $bug->priority === 'p1' ? 'error' : 'primary'
        ));
    }

    // 2. Notifikasi ke Developer Tertuju
    if (!empty($bug->developer)) {
        $dev = User::where('name', $bug->developer)->first();
        if ($dev && $dev->id !== $request->user()->id) {
            $dev->notify(new BugNotification(
                title: 'Penugasan Bug Baru',
                message: "Anda ditugaskan menangani bug #{$bug->id}: '{$bug->title}'",
                type: 'assigned', bugId: $bug->id, icon: 'person_add'
            ));
        }
    }
    return redirect()->route('bugs')->with('success', 'Bug reported successfully.');
```

```
public function updateStatus(Request $request, Bug $bug)
{
    $oldStatus = $bug->status;
    $bug->update($request->only('status', 'developer'));

    if ($oldStatus !== $bug->status) {
        // Notifikasi ke Pelapor Bug (Support Dev)
        if ($bug->reporter_id && $bug->reporter_id !== $request->user()->id) {
            $reporter = User::find($bug->reporter_id);
            if ($reporter) {
                $reporter->notify(new BugNotification(
                    title: 'Status Bug Diperbarui',
                    message: "{$request->user()->name} mengubah status bug #{$bug->id} menjadi '{$bug->status}'.",
                    type: 'status_updated', bugId: $bug->id,
                    icon: in_array($bug->status, ['fixed', 'closed']) ? 'check_circle' : 'sync'
                ));
            }
        }
    }
    return redirect()->route('bugs.show', $bug);
}
```

File: app/Models/User.php & Bug.php — Model User & Bug

Fungsi Inti: Model User mengaktifkan trait Notifiable untuk relasi polymorphic notifikasi; Model Bug memetakan relasi ke pelapor.

```
// app/Models/User.php
class User extends Authenticatable
{
    use HasFactory, Notifiable; // Notifiable menyediakan relasi notifications() & method notify()

    protected $fillable = ['name', 'email', 'password', 'role'];
}

// app/Models/Bug.php
class Bug extends Model
{
    protected $fillable = ['title', 'priority', 'status', 'developer', 'description', 'reporter_id'];

    public function reporter() {
        return $this->belongsTo(User::class, 'reporter_id'); // Relasi ke pembuat tiket
    }
}
```

File: resources/views/layouts/header.blade.php — Komponen Header Lonceng & Dropdown

Fungsi Inti: Mewujudkan antarmuka lonceng modern dengan pulse indicator, dropdown tab, dan interaktivitas AJAX.

```
<!-- resources/views/layouts/header.blade.php -->
<!-- 1. Trigger Ikon Lonceng dengan Animasi Pulse Merah -->
<button id="notification-bell-btn" type="button" class="relative p-2 rounded-xl text-secondary hover:text-primary">
    <span class="material-symbols-outlined text-[24px]">notifications</span>
    <span id="notification-badge" class="{{ $initialUnreadCount > 0 ? '' : 'hidden' }}" absolute top-1.5 right-1.5 flex
        <span class="animate-ping absolute inline-flex h-full w-full rounded-full bg-error opacity-75"></span>
        <span class="relative inline-flex rounded-full h-2.5 w-2.5 bg-error"></span>
    </span>
</button>

<!-- 2. Dropdown Panel Interaktif -->
<div id="notification-dropdown" class="hidden absolute right-0 mt-2 w-80 sm:w-[420px] bg-surface rounded-2xl shadow-2xl">
    <div class="p-4 border-b flex justify-between items-center">
        <h3 class="font-bold text-sm">Notifikasi</h3>
        <button id="mark-all-read-btn" class="text-xs text-secondary hover:text-primary">Tandai dibaca</button>
    </div>
    <div id="notification-list" class="max-h-[360px] overflow-y-auto divide-y">
        <!-- List notifikasi dirender dinamis via Blade dan AJAX -->
    </div>
</div>

<!-- 3. Skrip Interaktif AJAX & Real-time Click Handler -->
<script>
    function markAsRead(id, itemElement) {
        fetch(`/notifications/${id}/read`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', 'X-CSRF-TOKEN': csrfToken }
        }).then(res => res.json()).then(data => {
            if (itemElement) { itemElement.setAttribute('data-read', 'true'); }
            updateUnreadUI(data.unread_count); // Sembunyikan dot merah jika unread == 0
        });
    }

    // Polling background setiap 30 detik untuk deteksi notifikasi baru
    setInterval(() => { fetchNotifications(currentFilter); }, 30000);
</script>
```

File: tests/Feature/NotificationTest.php — Unit & Feature Testing Notifikasi

Fungsi Inti: Menguji fungsionalitas pengiriman otomatis notifikasi, otorisasi rute, dan status unread.

```
// tests/Feature/NotificationTest.php
class NotificationTest extends TestCase
{
    use RefreshDatabase;

    public function test_bug_creation_triggers_notifications_for_admin_and_developer(): void {
        $admin = User::factory()->create(['role' => 'super_admin']);
        $dev = User::factory()->create(['name' => 'dev1', 'role' => 'developer']);
        $support = User::factory()->create(['role' => 'support_dev']);

        $this->actingAs($support)->post(route('bugs.store'), [
            'title' => 'Critical Error', 'priority' => 'p1',
            'status' => 'open', 'developer' => 'dev1', 'description' => 'Test'
        ]);

        $this->assertGreaterThan(0, $admin->fresh()->unreadNotifications()->count());
        $this->assertGreaterThan(0, $dev->fresh()->unreadNotifications()->count());
    }

    public function test_status_update_triggers_notification_for_reporter(): void {
        $support = User::factory()->create(['role' => 'support_dev']);
        $dev = User::factory()->create(['role' => 'developer']);
        $bug = Bug::create([... 'reporter_id' => $support->id]);

        $this->actingAs($dev)->patch(route('bugs.status.update', $bug), ['status' => 'in_progress']);

        $this->assertEquals(1, $support->fresh()->unreadNotifications()->count());
    }
}
```